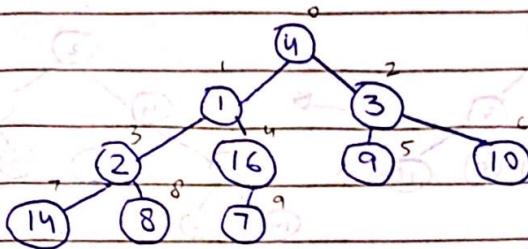


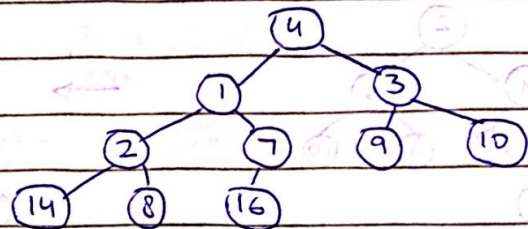
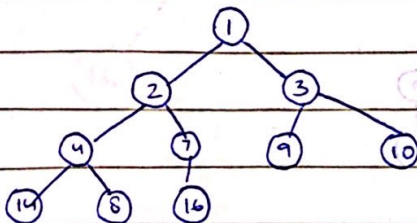
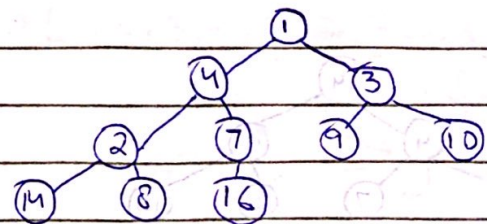
Assignment 2

Q1. $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$

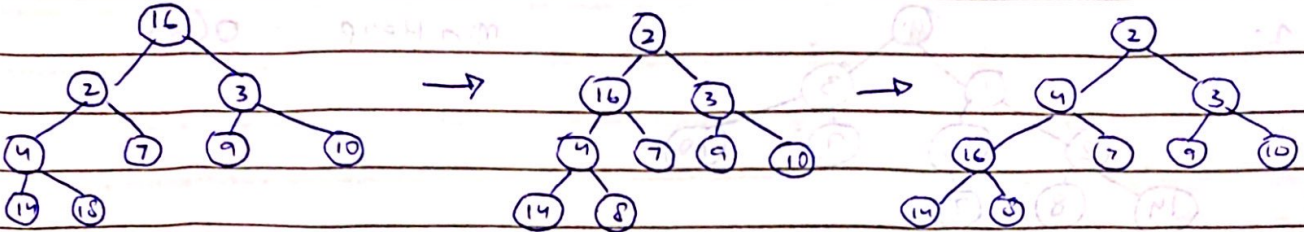
1.

Min Heap : $O(n)$

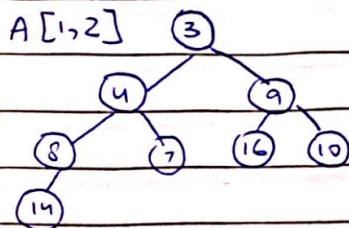
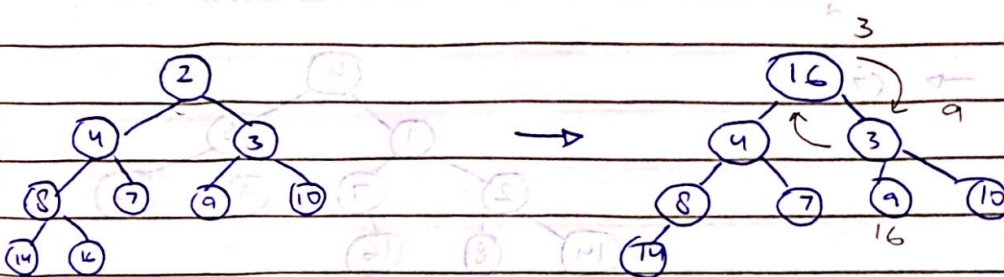
$$\frac{n}{2} - 1 = \frac{10}{2} - 1 = 4^{\text{th}} \text{ index}$$

 $A[4] = 16 \rightarrow (7)$  $A[3] = 2 \rightarrow (14, 8)$ no swap $A[2] = 3 \rightarrow (9, 10)$ no swap $A[1] = 1 \rightarrow (2, 7)$ no swap $A[0] = 4 \rightarrow (1, 3)$  $A = [1, 2, 3, 4, 7, 9, 10, 14, 8, 16]$

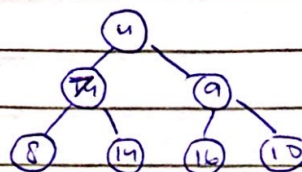
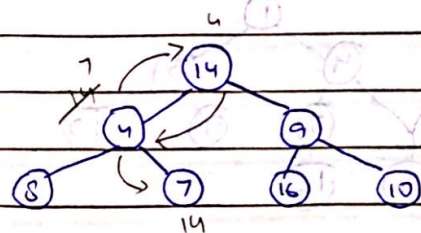
- remove root node, swap with rightmost leaf node
- swap 1 & 16



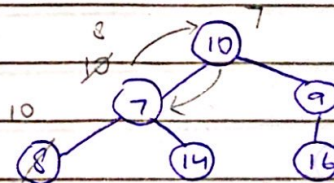
$A[1]$ = sorted

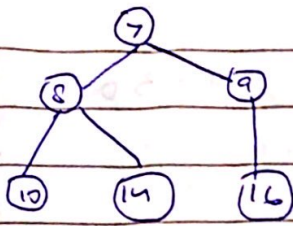


$A[1, 2, 3]$

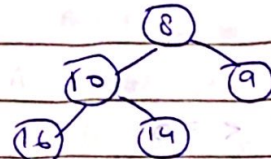
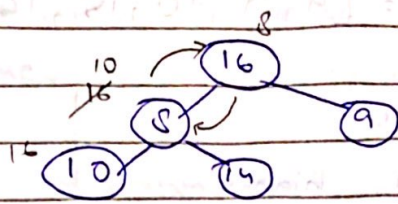


$A[1, 2, 3, 4]$



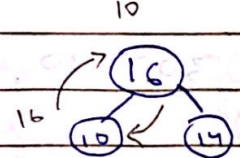
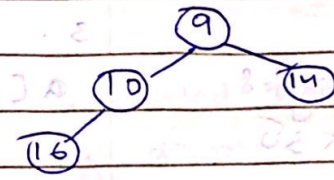
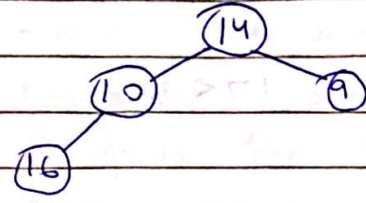


A [1, 2, 3, 4, 7]

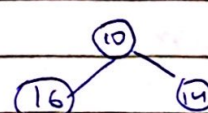


A [1, 2, 3, 4, 7]

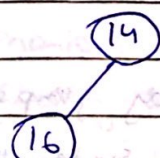
A [1, 2, 3, 4, 7, 8]



A [1, 2, 3, 4, 7, 8, 9]



A [1, 2, 3, 4, 7, 8, 9, 10]



A [1, 2, 3, 4, 7, 8, 9, 10, 14, 16]

Q2.

a) $arr = [4, 9, 10, 15, 16, 17, 21, 30]$ $x = 30$

Binary Search

1. $low = 1$ $mid = 4$ $high = 8$
 $a[4] = 15$ $15 < 30$

2. $low = 5$ $mid = 6$ $high = 8$
 $a[6] = 17$ $17 < 30$

3. $low = 7$ $mid = 7$ $high = 8$
 $a[7] = 21$ $21 < 30$

4. $low = 8$ $mid = 8$ $high = 8$
 $a[8] = 30$ $30 = 30$

found!

Time: $O(\log n)$

Space: $O(1)$

Adv: - efficiency on large array
- space efficient

Disadv: - required sorted array
- unnecessary complexity in smaller array

Best Case:

- Element in mid array
- large size array

Jump Search

Size = $\sqrt{8} = 2$
 $a[2] = 9$ $9 < 30$

$a[4] = 15$ $15 < 30$

$a[6] = 17$ $17 < 30$

$a[8] = 30$ $30 = 30$

found

Time: $O(\sqrt{n})$

Space: $O(1)$

Adv: - fine tuned by jump size
- efficient with system that store data in blocks

Disadv: - require sorted array
- require linear search

Best Case:

- element in end
- overhead linear search cost in small array

2b.

Interpolation : estimates position of target in sorted array based on its value, assuming uniform distribution

$$\text{pos} = \text{low} + \frac{x - a[\text{low}]}{a[\text{high}] - a[\text{low}]} \times (\text{high} - \text{low})$$

Best case $O(1)$ target element at 'pos' place

Avg ~~Best~~ case $O(\log \log n)$: uniformly distributed data

Worst case $O(n)$: non-uniform data

Scenario: Indexed databases

- works with uniform data

Exponential : Searching in unbounded / infinite length array.
It finds first a range by doubling step size & then performs binary search within the range

Best case : $O(1)$ element at $a[0]$

Avg case : $O(\log n)$: like binary search

Worst case : $O(\log n)$: Target element is far into array which requires full range

- not considered with uniform data

Scenario : ideal for searching in emails in archive emails.

Finding 1 in infinite size array

Q3. ID = 22K-5195

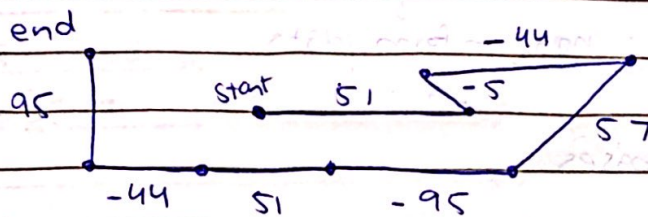
$$a = 51$$

$$b = 95$$

$$c = (51 \times 7) \bmod 60 = 57$$

$$d = 5195 \bmod 30 = 5$$

$$e = (95 - 51) = 44$$



Array A:

51	-5	-44	57	-95	51	-44	95
----	----	-----	----	-----	----	-----	----

Max subarray Sum (array) {

maxSum = -∞

currentSum = 0

for (i = 0 to n) {

currentSum += array[i]

if (currentSum < 0) {

currentSum = 0

start = i + 1

maxSum = max(maxSum, currentSum)

}

end = i

}

Dry run

i	a[i]	currentSum = 0	maxSum = ∞	start	end
0	51	51	51	0	0
1	-5	51 - 5 = 46	51	0	1
2	-44	46 - 44 = 2	51	0	2
3	57	57 + 2 = 59	59	0	3
4	-95	59 - 95 = -36 = 0	59	4	4
5	51	51	59	4	5
6	-44	51 - 44 = 7	59	4	6
7	95	7 + 95	102	10 4	7

Max profit is 102 sub array [57, -95, 51, -44, 95]
O(n) time.

Q4. 22K-5195

S = {51, 5195, 95, 19, 195, 22}

a = 1 c = 3 e = 5 g = 7 i = 9
b = 2 d = 4 f = 6 h = 8

C = {ea, eaie, ^{ie}ei, ai, ^{ai}ae, ~~ae~~}

Radix Sort O(n) least significant digit

longest string = eaie 4 = length

Counting Sort

[0, 10, 910, 99, 9109, 00]

1.

- Sort by last character (position 4)

ea = 1st = a
eaie → e
ie → e
ai → i
aie → e
bb → b

[ea, bb, eaie, ie, aie, ai]

2. Second last character (pos=3)

ea → e
bb → b
eaie → i
ie → i
aie → i
ai → a

[bb, ea, ai, eaie, ie, aie]

3. Third last character (pos=2)

bb → b
ea → a
ai → i
~~eaie~~
eaie → a
ie → e
aie → a

[bb, eaie, ea, aie, ai, ie]

4. First character (pos=1)

bb → b

eaie → e

ea → e

aie → a

ai → a

ie → i

Final = [aie, ai, bb, ea, eaie, ie]

5.

~~A [1, 0, 4, 0, 0, 0, 2, 0]~~

~~A = [1, 0, 4, 0, 0, 0, 2, 0]~~

~~A = [~~

int shiftarray (int[] a) {

int pos = 0

for (int i = 0; i < a.length; i++) {

if (a[i] != 0 {

a[i] = a[i] + a[pos]

a[pos] = a[i] - a[pos]

a[i] = a[i] - a[pos]

pos++;

}

}

return a;

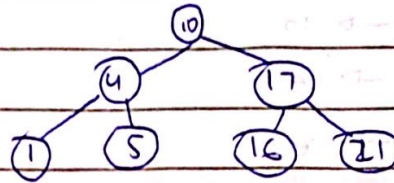
}

$O(n)$ - TC

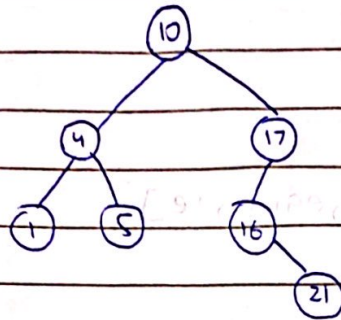
$O(1)$ - SC

Q6.

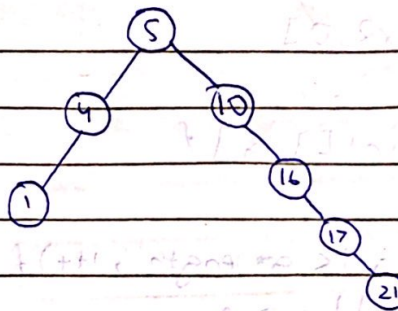
* Height = 2



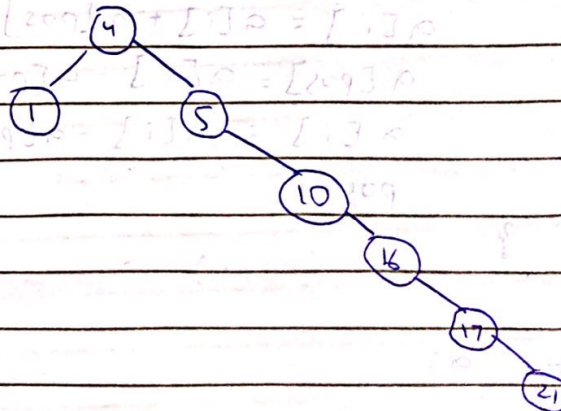
* h = 3



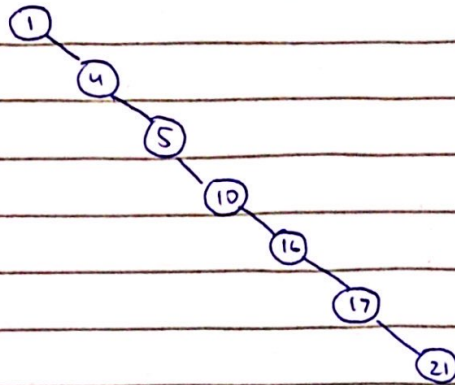
* Height = 4



* Height = 5



* Height = 6



Q7.

FindDuplicate (int [] a) {

HashMap < Integer, Integer > b = new HashMap<>();

for (int i = 0; i < a.length; i++) { —— n+1

if (b.containsKey(num[i])) { —— n

b.put(num, b.get(num)+1); —— n

}

else {

b.put(num, 1); —— n

}

}

boolean found = false; —— 1

for (int key: b.keySet()) { —— n+1

if (b.get(key) > 1) { —— n

System.out.print(key); —— n

found = true; —— n

}

}

$8n+3$

$O(n)$